

Forcebit Ticketing

Een support ticketing applicatie met focus op functionaliteit, workflow en onderhoudbare structuur

Student	Alex Delvoye
Opleiding	Graduaat Programmeren
Academiejaar	2025-2026
Project	Forcebit Ticketing
Versie	Eerste inhoudelijke draft

Samenvatting

Voor mijn graduaatsproef werkte ik aan een ticketing systeem voor Forcebit. Het doel van de applicatie is om communicatie tussen klanten en support duidelijker te maken. Een klant kan een ticket aanmaken, antwoorden sturen, bestanden toevoegen en de status opvolgen. Een administrator kan alle tickets bekijken, filteren, beantwoorden en de status aanpassen.

In dit rapport leg ik vooral uit hoe de functionaliteit is opgebouwd en waarom de code in verschillende lagen is verdeeld. De nadruk ligt op Single Responsibility, de meerlagenstructuur, de ticket workflow en de frontend logica. Ik probeer dit bewust uit te leggen op het niveau waarop ik het zelf kan verdedigen: niet als senior engineer, maar als graduaatstudent die keuzes heeft gemaakt om de applicatie overzichtelijk en uitbreidbaar te houden.

1. Inleiding en leeswijzer

Deze eerste draft is bedoeld als basis voor het uiteindelijke graduaatsproefrapport. De tekst focust nog niet op elk klein codefragment. Die details staan al in de README en technische README van het project. Dit rapport legt vooral uit wat de applicatie doet, hoe de logica door de lagen loopt en waarom de structuur helpt om het project te begrijpen.

Ik heb het project stap voor stap opgebouwd vanuit een praktische nood: supportvragen moeten niet verloren gaan in losse mails, chatberichten of mondelinge afspraken. Een ticket moet een duidelijke plaats hebben, met een titel, categorie, onderwerp, status, berichten en eventueel bijlagen. Daardoor kan zowel de klant als de administrator volgen wat er al besproken is.

Inhoud

1. Inleiding en leeswijzer	2
2. Probleemstelling en doel	3
3. Functionaliteit voor klant en administrator	4
4. Gebruikte technologieën	5
5. Backend structuur en Single Responsibility	6
6. Services, domain rules en ticket workflow	7
7. Persistence, database en startupcontrole	8
8. Frontend structuur: screens, hooks en components	9
9. Dashboardlogica: zoeken, filteren en paginatie	10
10. Bijlagen, previews, e-mail en beveiliging	11
11. Validatie, foutafhandeling en logging	12
12. Kwaliteit, documentatie en testbaarheid	13
13. Reflectie en toekomstig werk	14

Schrijfstijl. Dit rapport is bewust geschreven als een studentendocument. Ik gebruik technische termen waar nodig, maar ik probeer telkens ook in gewone taal uit te leggen waarom iets zo is gebouwd.

2. Probleemstelling en doel

Het probleem dat dit project probeert op te lossen is herkenbaar in veel kleine organisaties. Klanten hebben vragen of problemen, maar de opvolging gebeurt vaak via losse communicatiekanalen. Daardoor wordt het moeilijk om te zien welke problemen nieuw zijn, welke al besproken worden en welke opgelost zijn.

Een ticketing systeem brengt die communicatie samen. Een ticket is dan niet alleen een vraag, maar ook de volledige geschiedenis van die vraag. Berichten, statuswijzigingen en bijlagen blijven op een centrale plaats staan. Voor support is dat belangrijk omdat meerdere tickets tegelijk opgevolgd moeten worden. Voor klanten is het belangrijk omdat ze kunnen zien wat de huidige stand van zaken is.

Doel van de applicatie

Het doel was om een bruikbare basisapplicatie te bouwen waarin twee rollen samenwerken: klant en administrator. De klant moet zonder technische kennis een ticket kunnen aanmaken en opvolgen. De administrator moet snel kunnen zoeken, filteren en antwoorden zonder door een lange ongestructureerde lijst te moeten scrollen.

Technische leerdoelen

- Een backend bouwen met duidelijke lagen en verantwoordelijkheden.
- Business rules niet alleen in de frontend plaatsen, maar ook in de backend afdwingen.
- Een frontend bouwen met herbruikbare hooks, componenten en utilities.
- Een workflow ontwerpen die logisch blijft wanneer tickets gesloten of heropend worden.
- Documentatie schrijven waarmee ik de keuzes kan uitleggen en verdedigen.

Afbakening

Het project is geen volledig commercieel ticketsysteem. Zaken zoals uitgebreide rapportage, SLA's, rollenbeheer en productiehosting vallen buiten deze eerste versie. De focus ligt op de kern: tickets, conversaties, statussen, bijlagen, dashboards en een onderhoudbare structuur.

3. Functionaliteit voor klant en administrator

Forcebit Ticketing bestaat uit functionaliteit voor twee soorten gebruikers. De klant gebruikt de applicatie vooral om problemen te melden en op te volgen. De administrator gebruikt de applicatie als supportdashboard.

Functionaliteit voor klanten

Een klant kan zich registreren, inloggen en een profiel beheren. Daarna kan de klant een ticket aanmaken. Bij het aanmaken kiest de klant een categorie en onderwerp, vult een titel in en schrijft de eerste boodschap. Die eerste boodschap wordt opgeslagen als eerste bericht in de conversatie. Dat maakt de opbouw eenvoudiger: alle tekstberichten staan chronologisch in dezelfde berichtenlijst.

Een klant kan antwoorden op een ticket zolang het niet gesloten is. Daarnaast kan de klant bijlagen toevoegen, zoals screenshots, PDF-bestanden of zip-bestanden. Afbeeldingen van het type PNG, JPG en JPEG krijgen een preview zodat de gebruiker ze niet eerst moet downloaden. De klant kan een ticket sluiten wanneer het probleem opgelost is en opnieuw openen als het probleem toch nog bestaat.

Functionaliteit voor administrators

Een administrator ziet een overzicht van klanten en tickets. Omdat een lange lijst snel onduidelijk wordt, zijn er zoekvelden, filters en paginatie toegevoegd. Tickets kunnen gezocht worden op titel, klant, categorie en onderwerp. Daarnaast kan de administrator filteren op status, categorie en onderwerp.

De tickets zijn gegroepeerd in drie delen: New, Open en Closed. New betekent dat support nog niet geantwoord heeft. Open betekent dat de conversatie actief is. Closed betekent dat het ticket afgehandeld is. Deze verdeling maakt het dashboard bruikbaar dan een lange lijst zonder duidelijke prioriteit.

Belangrijkste meerwaarde: het project brengt ticketcommunicatie, statusopvolging, bijlagen en adminbeheer samen in een duidelijke workflow.

4. Gebruikte technologieën

De backend is gebouwd met ASP.NET Core en draait als Web API. De database is MySQL en de toegang tot de database gebeurt met Entity Framework Core. De frontend is gebouwd met Expo React Native, zodat dezelfde code ook op web kan draaien. Voor formulieren worden Formik en Yup gebruikt.

Backend

- ASP.NET Core Web API
- Entity Framework Core
- MySQL via Pomelo provider
- JWT authenticatie
- Brevo voor e-mailnotificaties

Frontend

- Expo React Native
- React Navigation
- TypeScript
- Formik en Yup
- Herbruikbare hooks en componenten

Waarom deze stack?

ASP.NET Core past goed bij een gestructureerde API met controllers, dependency injection, middleware en typed options. Entity Framework Core maakt het mogelijk om met C# entiteiten te werken en toch migraties naar MySQL te beheren. React Native met Expo past bij het frontendgedeelte omdat de applicatie op web kan draaien, maar de structuur ook dicht bij mobile development blijft.

TypeScript helpt om frontend types duidelijk te maken. Dat is belangrijk omdat statussen, categorieën en API responses overal dezelfde betekenis moeten hebben. Formik en Yup houden formulierstatus en validatie uit de screenbestanden, waardoor de schermen minder zwaar worden.

Projectstructuur

De backend is verdeeld in vier projecten: API, Services, Persistence en Domain. De frontend is verdeeld in folders zoals screens, hooks, forms, components, api, styles, utils, validation en types. Deze verdeling helpt mij om tijdens het ontwikkelen sneller te weten waar nieuwe code hoort.

5. Backend structuur en Single Responsibility

Een belangrijk principe in dit project is Single Responsibility. Voor mij betekent dit dat een klasse of module een duidelijke hoofdtak moet hebben. Dat maakt de code gemakkelijker om uit te leggen en om later aan te passen.

API laag

De API laag ontvangt HTTP requests. Controllers lezen routeparameters, body data, formulierdata en JWT claims. Daarna roepen ze een service aan. Een controller beslist niet zelf hoe de volledige ticket workflow werkt. Dat zou de controller te groot maken en de logica verspreiden.

Bij uploads gebruikt de API laag specifieke form request classes. Die bestaan omdat multipart form-data een HTTP-detail is. De service krijgt geen `IFormFile`, maar een gewone `FileUploadRequest`. Zo blijft ASP.NET-specifieke code aan de rand van het systeem.

Services laag

De services laag bevat de toepassinglogica. `TicketService` is daar een goed voorbeeld van. Deze service maakt tickets aan, laadt ticketdetails, voegt berichten toe, verandert statussen, bewaart bijlagen en beslist wanneer er e-mail moet worden verstuurd. De service kent wel de workflow, maar niet de details van HTTP responses.

Domain en persistence

De domain laag bevat de concepten van de applicatie. Een ticket heeft een status, berichten, bijlagen en regels. De persistence laag is verantwoordelijk voor EF Core en MySQL. Repositories verbergen de concrete queries voor de services. Daardoor moet een service niet telkens zelf weten welke `Include` nodig is om berichten, bijlagen en gebruikers mee te laden.

Deze verdeling is niet perfect theoretisch, maar ze is voor dit project duidelijk genoeg. Ik kan per map uitleggen waarom de code daar staat.

6. Services, domain rules en ticket workflow

De belangrijkste workflow in de applicatie is de levenscyclus van een ticket. Op basis van feedback is de workflow duidelijker gemaakt met de statussen `New`, `Open` en `Closed`.

Statussen

- **New:** het ticket is nieuw en support heeft nog niet geantwoord.
- **Open:** de conversatie is actief of een gesloten ticket werd opnieuw geopend.
- **Closed:** het ticket is afgehandeld en er kan niet meer op geantwoord worden.

`New` is bewust alleen voor nieuw aangemaakte tickets. Een gebruiker kan een bestaand ticket niet terug op `New` zetten. Dat zou verwarrend zijn, omdat een bestaand ticket al geschiedenis heeft. Wanneer een gesloten ticket opnieuw geopend wordt, gaat het naar `Open` en niet naar `New`.

Rol van TicketService

`TicketService` voert de use cases uit. Bij ticketcreatie controleert de service eerst of de klant bestaat en of categorie en onderwerp geldig zijn. Daarna maakt de domain entity het ticket aan. De eerste boodschap wordt toegevoegd als eerste `TicketMessage`. Bij antwoorden controleert de service toegang en statusregels voor het bericht wordt opgeslagen.

Rol van TicketRules

`TicketRules` bevat regels zoals: wie mag een ticket bekijken, wie mag antwoorden en welke statuswijzigingen zijn toegelaten. Dit is belangrijk omdat de frontend niet de enige bescherming mag zijn. Een gebruiker kan altijd proberen om rechtstreeks de API aan te spreken. Daarom moet de backend de regels opnieuw controleren.

Waarom status automatisch verandert

Wanneer een administrator antwoordt op een `New` ticket, verandert het ticket automatisch naar `Open`. Dat past bij de betekenis van `Open`: support is ermee bezig. Deze overgang zit in de backendlogica, zodat de status ook correct verandert als een andere client dan de frontend de API gebruikt.

7. Persistence, database en startupcontrole

De persistence laag bevat de databasecontext, configuraties, repositories en migraties. Entity Framework Core vertaalt de C# entiteiten naar tabellen in MySQL. De migraties bewaren hoe het datamodel doorheen de ontwikkeling veranderd is.

Repositories

Repositories zorgen ervoor dat services niet rechtstreeks met EF Core queries moeten werken. Bijvoorbeeld: voor een ticketdetail moeten ook berichten, afzenders, klantgegevens en bijlagen mee geladen worden. Door dat in een repositorymethode te plaatsen, blijft de service meer gefocust op de workflow.

DTOs tegenover entiteiten

De API stuurt geen volledige EF entiteiten naar de frontend. In plaats daarvan worden DTOs gebruikt. Een lijstitem bevat minder data dan een detailresponse. Dat is nuttig omdat overzichten niet alle berichten en bijlagen nodig hebben. Het maakt de API response ook stabielere wanneer de database intern verandert.

Startupcontrole

De backend controleert bij startup of MySQL bereikbaar is. Dat is bewust fail-fast gedrag. Zonder database kunnen login, tickets, berichten, bijlagen en de admin seed niet betrouwbaar werken. Een duidelijke startupfout is beter dan een API die opstart maar pas faalt wanneer iemand probeert in te loggen.

Admin seed

Voor development en demo's wordt een admin account uit configuratie aangemaakt als het nog niet bestaat. De seedlogica hoort bij startup en niet bij normale ticketfunctionaliteit. Daardoor blijft het duidelijk dat dit vooral demo- en ontwikkelgemak is.

8. Frontend structuur: screens, hooks en components

In de frontend heb ik geprobeerd om schermen niet te zwaar te maken. Een screen moet vooral tonen wat er op de pagina staat. Het gedrag van een pagina staat zoveel mogelijk in hooks. Daardoor zijn bestanden zoals de homepagina, adminpagina en ticketdetailpagina gemakkelijker te lezen.

Screens

Screens zijn page-level componenten. Ze combineren formulieren, knoppen, lijsten, headers en components. Een screen mag weten hoe de pagina eruit ziet, maar niet alle details van data ophalen, filteren of verzenden hoeven daarin te staan.

Hooks als gedragslaag

`useHomeScreen` beheert de tickets van een klant. Deze hook laadt tickets, bewaart zoektekst en filters, berekent groepen en geeft data terug aan het scherm. `useAdminScreen` doet iets gelijkaardigs voor administrators, maar met extra filters voor klant, status, categorie en onderwerp.

Het ticketdetail scherm gebruikt `useTicketDetailScreen`. Die hook laadt het ticket, verzendt antwoorden, verandert statussen, downloadt bijlagen en maakt image previews aan. De screen component moet daardoor niet zelf alle API details kennen.

Components en utilities

Herbruikbare UI staat in components. Voorbeelden zijn `AppHeader`, `PaginationControls` en `TicketGroupSection`. Utilities bevatten herbruikbare bewerkingen zoals ticket search, ticket grouping, datumformatting en attachment form-data. Hierdoor is de logica niet verspreid over meerdere schermen.

Types

TypeScript types helpen om frontend en backend op elkaar af te stemmen. De ticketstatussen in de frontend zijn bijvoorbeeld letterlijk `New`, `Open` en `Closed`. Dat voorkomt typfouten zoals "In progress" en maakt autocomplete nuttig tijdens ontwikkeling.

9. Dashboardlogica: zoeken, filteren en paginatie

De dashboards zijn aangepast omdat lange lijsten snel onbruikbaar worden. Eerst was de clientlijst vooral een verzameling knoppen. Dat werkte technisch, maar niet goed wanneer er veel klanten zijn. Daarom zijn zoeken, filtering, visuele tellingen en paginatie toegevoegd.

Derived state

Een belangrijk frontendprincipe in dit project is derived state. De applicatie bewaart de ruwe data uit de API, bijvoorbeeld alle tickets. Zoekresultaten, filters, groepen en tellingen worden daarvan afgeleid. Dat is veiliger dan meerdere aparte lijsten bijhouden, want aparte lijsten kunnen uit sync raken.

Admin dashboard pipeline:

```
clients + tickets -> searched clients -> filtered tickets -> grouped sections -> paginated section items
```

Client dashboard pipeline:

```
tickets -> search/filter -> grouped New/Open/Closed sections -> paginated section items
```

Zoeken en filteren

Klanten kunnen hun tickets zoeken en filteren op status en categorie. Administrators kunnen klanten zoeken op naam, bedrijfsnaam of e-mail. Tickets kunnen op de adminpagina gezocht worden op titel, klant, categorie en onderwerp. Daarnaast zijn er filters voor status, categorie en onderwerp.

Paginatie

Voor paginatie is een algemene `usePagination` hook gemaakt. `PaginationControls` toont de knoppen en tekst. `TicketGroupSection` combineert ticketgroepen met paginatie. Daardoor gebruiken de admin- en klantpagina dezelfde basislogica. Elke groep heeft eigen paginatie, zodat bladeren in Closed tickets geen invloed heeft op New of Open tickets.

10. Bijlagen, previews, e-mail en beveiliging

Bijlagen zijn belangrijk in een ticketing systeem omdat klanten vaak screenshots, documenten of zip-bestanden meesturen. In deze applicatie kunnen bijlagen toegevoegd worden bij ticketcreatie en bij antwoorden.

Uploadflow

De frontend gebruikt een attachment picker. Gebruikers kunnen meerdere keren bestanden toevoegen voor ze verzenden. Ze kunnen een bestand apart verwijderen of alles wissen. De frontend toont ook hoeveel van de 20 MB uploadlimiet al gebruikt is.

De backend controleert dezelfde limiet opnieuw. Dat is belangrijk omdat frontendcontrole alleen niet veilig genoeg is. De service bewaart het bestand via de file storage service en maakt metadata aan in de database.

Previews en downloads

PNG, JPG en JPEG bestanden krijgen een inline preview in de conversatie. De bestanden worden niet publiek bereikbaar gemaakt. De frontend haalt de preview op via een beschermde API route met JWT. Daarna maakt de browser tijdelijk een object URL. Oude object URLs worden opgeruimd zodat de browser geen geheugen blijft vasthouden.

E-mailnotificaties

Brevo wordt gebruikt om klanten te verwittigen wanneer een administrator antwoordt of de status wijzigt. Er worden geen e-mails naar administrators gestuurd wanneer klanten antwoorden. De reden is dat het dashboard de werkqueue van support is. Zo wordt de mailbox minder snel overspoeld.

Beveiliging

Downloads en previews gebruiken dezelfde tickettoegang als de rest van de applicatie. Een klant mag alleen eigen tickets en bijlagen zien. Administrators mogen alle tickets zien. Die controle hoort in de backend, omdat de frontend niet als beveiligingslaag beschouwd mag worden.

11. Validatie, foutafhandeling en logging

De applicatie gebruikt verschillende soorten validatie. In de frontend zorgt Yup ervoor dat gebruikers snel feedback krijgen. In de backend blijven DTO-validatie, services en domain rules de echte bescherming. Frontendvalidatie is goed voor gebruiksgemak, maar backendvalidatie is nodig voor veiligheid en betrouwbaarheid.

Authenticatie en autorisatie

Gebruikers loggen in met e-mail en wachtwoord. Wachtwoorden worden niet als platte tekst bewaard, maar gehasht. Na login krijgt de frontend een JWT-token. Bij volgende API calls wordt dat token meegestuurd. De backend leest daaruit de user id en rol.

Autorisatie gebeurt op twee niveaus. Sommige routes zijn beschermd met rollen, bijvoorbeeld admin routes. Specifieke ticketregels staan in `TicketRules`. Zo kan een klant alleen eigen tickets bekijken, terwijl een administrator alle tickets kan zien.

Foutafhandeling

De backend gebruikt middleware voor centrale foutafhandeling. Services gooien betekenisvolle exceptions zoals `NotFound`, `Forbidden` of `BadRequest`. De middleware vertaalt die naar een consistente JSON response. Daardoor hoeven controllers niet overal dezelfde try/catch code te schrijven.

De frontend normaliseert API-fouten en toont duidelijke meldingen. Toast notifications geven feedback na acties zoals ticket aanmaken, antwoord verzenden of status wijzigen. Formulierfouten blijven dicht bij het veld staan.

Logging

De backend logt belangrijke acties zoals tickets aanmaken, berichten toevoegen en statussen wijzigen. Structured logging maakt het gemakkelijker om waarden zoals ticket id, user id en status terug te vinden. Tijdens ontwikkeling en demo's helpt dit om te begrijpen wat de backend precies gedaan heeft.

12. Kwaliteit, documentatie en testbaarheid

Een belangrijk deel van dit project is niet alleen dat de applicatie werkt, maar ook dat ik kan uitleggen hoe ze werkt. Daarom zijn comments en documentatie mee onderhouden. De gewone README bevat vooral installatie en gebruik. De technische README legt architectuur, workflows en keuzes uit.

Comments

Ik heb geprobeerd comments niet te gebruiken om elke regel code te herhalen. Een comment moet vooral uitleggen waarom iets gebeurt of waarom een keuze gemaakt is. Bijvoorbeeld: waarom de eerste ticketbeschrijving een bericht wordt, waarom New creation-only is, of waarom uploaded files eerst naar een service DTO gemapt worden.

Consistentie

Consistentie is belangrijk omdat het project meerdere schermen en workflows heeft. Statusnamen moeten overall hetzelfde betekenen. De frontendtypes spiegelen daarom de backendstatussen. De ticketlijsten op de klant- en adminpagina gebruiken dezelfde groepstructuur. Ook de uploadlimiet wordt op frontend en backend bewaakt.

Testbaarheid

Er zijn nog niet genoeg automatische tests om het project production-ready te noemen. Toch helpt de structuur wel om later tests toe te voegen. Services kunnen getest worden rond use cases. Domain rules kunnen getest worden zonder HTTP. Repositories kunnen getest worden met databasegerichte tests. Frontend hooks en helpers kunnen apart getest worden voor filtering en paginatie.

Handmatige controle

Tijdens de ontwikkeling zijn linting, typechecking en backend builds gebruikt om fouten sneller te vinden. Voor functionele controle is de applicatie met testdata gebruikt: veel klanten, veel tickets, zoektermen, filters en langere conversaties. Dat was nodig om te zien of de paginatie en layout ook bruikbaar blijven met meer data.

13. Reflectie en toekomstig werk

Ik heb in dit project vooral geleerd dat structuur belangrijker wordt naarmate een applicatie groeit. In het begin lijkt het sneller om logica rechtstreeks in een controller of screen te plaatsen. Later wordt dat moeilijker om te begrijpen. Door services, hooks, repositories, domain rules en utilities apart te houden, blijft het project beter verdedigbaar.

Wat goed gelukt is

De ticketworkflow is duidelijker geworden door de statussen New, Open en Closed. De chatweergave maakt de conversatie makkelijker leesbaar. De admin- en klantdashboards zijn bruikbaar geworden door zoekvelden, filters, groepen en paginatie. Ook de bijlageflow is praktischer geworden door preview voor afbeeldingen en een duidelijke uploadlimiet.

Wat ik anders zou aanpakken

Als ik opnieuw zou starten, zou ik vroeger automatische tests toevoegen. Ik zou ook vroeger nadenken over server-side paginatie. Voor deze versie is client-side paginatie logisch omdat de lijsten nog beperkt zijn, maar in een echte productieomgeving kan de hoeveelheid tickets snel groeien.

Toekomstig werk

- Automatische integratietests voor de belangrijkste workflows.
- Server-side paginatie voor grote ticket- en klantlijsten.
- Betere productieconfiguratie voor secrets en logging.
- Uitgebreider adminbeheer voor rollen en gebruikers.
- Meer rapportage, bijvoorbeeld tickets per categorie of gemiddelde doorlooptijd.

Conclusie

Forcebit Ticketing is een functionele ticketing applicatie met een duidelijke basisstructuur. De applicatie ondersteunt klanten en administrators in een realistische supportflow. De technische keuzes zijn niet gekozen om complex te zijn, maar om de code begrijpbaar te houden: HTTP in controllers, workflows in services, business rules in de domain laag, databasecode in repositories en schermgedrag in frontend hooks.

Voor mij is dit project vooral een oefening geweest in bewust structureren. Ik begrijp beter waarom Single Responsibility en een meerlagenstructuur nuttig zijn: niet omdat ze mooi klinken, maar omdat ze helpen om later uit te leggen, te verbeteren en veilig aan te passen.